

Cryptography Homework 3

Piter Zac. Garcia B.
Vasudev Prasad Bethamcherla

Due Date: Monday, 6/24 10:00AM

1. Exercise 6.8.10. The exponents $e=1$ and $e=2$ should never be used in RSA.

Why?

Because if you use either $e = 1$ or $e = 2$ is very easy for anyone that intercept the RSA cipher-text to figure out what the plaintext is, for example:

If you pick $e = 1$, we have the following situation:

$C = m^e \bmod n = m^1 \bmod n = m \bmod n$ it is way obvious that whoever has the cipher-text will have the plaintext as well since they both the same. The user would notice so because the cipher-text would legible.

If you pick $e = 2$, we have the following situation:

$$C = m^e \bmod n = m^2 \bmod n$$

Now, to find the key "d" we need to find $\gcd(e, \Phi(n)) = 1$. However, $\gcd(2, \Phi(n)) = 2$ which will make it impossible for anyone to find a key.

Due to the above situation, "*e must always be a odd number*".

2. Exercise 6.8.21 (page 195): Let $p=7919$ and $q=17389$. Let $e=66909025$. A calculation shows that $e^2 = 1 \pmod{(p-1)(q-1)}$. Alice decides to encrypt the message $m=12345$ using RSA with $n=pq$ and exponent e . Since she wants the encryption to be very secure she encrypts the message twice again with n and e .

What is the final ciphertext she sends?

$$C = (m^e)^e = m^{e^2} = (12345)^{(66909025)^2}$$

Using a RSA.java program for faster and large computation we get the following:

Output Code:

For $n = 137703491$

Measure of Integers that Share the Factor of 1: 137678184

$N = PQ = 137703491$

$P = 7919$

$Q = 17389$

$d = 264049225$

Message is:

$C = 12345$

$e = 66909025$

message = 12345

encrypted = 92626159

decrypted = 12345

message = 92626159

encrypted = 12345

decrypted = 92626159

Explain why it is so.

Enc_{pk} is deterministic, so:

If we encrypt twice the same message with the same key, then the cipher-texts and the plaintext are the same. Therefore if the message space m is small, the adversary can check all possible messages:

Calculation Proof:

$$m^k \Phi(n) = 1 \bmod n \rightarrow m * m^k \Phi(n) = m \bmod n \rightarrow m^k \Phi(n) + 1 = m \bmod n$$
$$m^{ed} = m \bmod n \rightarrow d = (k \Phi(n) + 1) / e$$

Following the above calculation with e^2 we get the following:

$$c = (m^e)^e = m^{e^2} = (m * m)^e = m \bmod n \rightarrow m^e + 1 = m \bmod n$$

$$ed = (k \Phi(n) + 1) \rightarrow \text{in this case, } e+1 = (k \Phi(n) + 1).$$

Note that $d = 1$, and therefore the cipher-text is the same as the plaintext since

$$c^d = c^1 = c = m, \text{ which says the cipher-text is equal to the message.}$$

Source Code: For a better reference to the source code look at the Gcd.java file

```
// N-bit (roughly) public and private key
Gcd(int N) {
    BigInteger p = BigInteger.valueOf(7919);
    BigInteger q = BigInteger.valueOf(17389);
    BigInteger phi =
        (p.subtract(1)).multiply(q.subtract(1));

    modulus = p.multiply(q);
    publicKey = new BigInteger("66909025");
    privateKey = publicKey.modInverse(phi);
}

public Gcd(Integer a, Integer b) {
    setGcd(gcd(a, b));
}

public static void main(String[] args) {

    int e = 66909025;
    int[] m = {12345};
    int p = 7919;
    int q = 17389;
    int n = p*q;
    int phi = (p-1)*(q-1);
    System.out.println("For n = " + n + "" + "\nMeasure
of Integers that Share the Factor of 1: " + phi);
    System.out.println("N = PQ = " + pq(n, phi));
    int d = findD(phi, e);
    System.out.println("P = " + p);

    System.out.println("Q = " + q);
    System.out.println("d = " + d);
    System.out.println("Message is: ");
    System.out.println("C = " + m[0]);

    Gcd key = new Gcd(n);
    System.out.println(e);
    BigInteger message = BigInteger.valueOf(12345);
    BigInteger encrypt = key.encrypt(message);
    BigInteger decrypt = key.decrypt(encrypt);
    System.out.println("message = " + message);
    System.out.println("encrypted = " + encrypt);
    System.out.println("decrypted = " + decrypt);

    BigInteger encrypt1 = key.encrypt(encrypt);
    BigInteger decrypt1 = key.decrypt(encrypt1);
    System.out.println("message = " + encrypt);
    System.out.println("encrypted = " + encrypt1);
    System.out.println("decrypted = " + decrypt1);
}

public BigInteger encrypt(BigInteger message) {
    return message.modPow(publicKey, modulus);
}

public BigInteger decrypt(BigInteger encrypted) {
    return encrypted.modPow(privateKey, modulus);
}

BigInteger decrypt(BigInteger encrypted) {
    return encrypted.modPow(privateKey, modulus);
}
```

3. Exercise 6.8.15 (page 194): Suppose n is a large odd number. You calculate $2^{(n-1)/2} = k \pmod{n}$, where k is not equal to $\pm 1 \pmod{n}$.

○ **Suppose $k^2 \not\equiv 1 \pmod{n}$. Explain why this implies that n is not prime.**

You calculate $2^{(n-1)/2} = k \pmod{n}$, where k is not equal to $\pm 1 \pmod{n}$.

Using the Fermat's little theorem (page 79): if p is prime, then $k^{p-1} = 1$. This is because there are composite numbers n such that $k^{(n-1)} = 1 \pmod{n}$ for all $1 < a < n-1$.

Proof:

$$k = 2^{(n-1)/2} \rightarrow k^2 = (2^{(n-1)/2})^2 = 2^{(n-1)2/2} = 2^{(n-1)} \rightarrow k^2 \not\equiv 1 \pmod{n} \text{ is } 2^{(n-1)} \not\equiv 1 \pmod{n}$$

To determine if n is composite, find a non-trivial square root of 1 mod n (root of unity).

Suppose that $x^2 \equiv 1 \pmod{n}$ but $x \not\equiv 1 \pmod{n}$. Then n is composite.

If we have, $x = 2^{(n-1)/2}$ then $(2^{(n-1)/2})^2 \not\equiv 1$ since $2^{(n-1)} \not\equiv 1$

Proof:

$$x^2 = (2^{(n-1)/2})^2 = 2^{(n-1)} = 2^i, \text{ where } i > 0 \text{ since } x = 2^{(n-1)/2} \not\equiv 1 \pmod{n},$$

Therefore $x = 2^{(n-1)/2} \not\equiv 1$, and $x^2 \not\equiv 1$ by the root of unity.

Both witness prove that n , where $k = 2^{(n-1)/2}$, is a composite number. Therefore, using Fermat's little theorem and Monte Carlos theorem we have proved that the number is not prime but instead composite.

○ **Suppose $k^2 \equiv 1 \pmod{n}$. Explain how you can use this to factor n .**

If p is prime, then $k^{p-1} = 1$. This is because there are composite numbers n such that $k^{(n-1)} = 1 \pmod{n}$ for all $1 < a < n-1$.

Proof:

$$k = 2^{(n-1)/2} \rightarrow k^2 = (2^{(n-1)/2})^2 = 2^{(n-1)2/2} = 2^{(n-1)} \rightarrow k^2 \equiv 1 \pmod{n} \text{ is } 2^{(n-1)} \equiv 1 \pmod{n}$$

Suppose that $x^2 \equiv 1 \pmod{n}$ but $x \not\equiv 1 \pmod{n}$. Then n is composite.

If we have, $x = 2^{(n-1)/2}$ then $(2^{(n-1)/2})^2 \equiv 1$ since $2^{(n-1)} \equiv 1$

Proof:

If $x = 2^{(n-1)/2} \equiv 1 \pmod{n}$, then the square root of 1 will always be one.

Therefore $x = 2^{(n-1)/2} \equiv 1$, and $x^2 \equiv 1$ by the root of unity.

Both witness prove that n , where $k = 2^{(n-1)/2}$, is a prime number. Therefore, n is easier to factor out since the factor of n is $n-1$.

4. In the notes, we claimed that the error probability of Miller-Rabin is at most 1/4. You will now test this claim (a little).

○ **Find the largest error probability of Miller-Rabin for integers between 100,000 and 101,000 (inclusive).**

Please use the Miller-Rabin version from the notes. It is ok to completely brute-force. In your answer, you should give this largest error probability (with lots of decimals) and the integer between 100,000 and 101,000 (inclusive) for which this largest error probability occurs. If there is more than one such integer, you should give the smallest such integer.

highest Probabilities According to its Percentage and Lowest Value:

[100003 NP: 0 P: 250.25, 100001 NP: 0.5 P: 0, 100009 NP: 0.25 P: 0.25, 100000 NP: 0.25 P: 0]
[100003 NP: 0 P: 250.25, 100081 NP: 0.5 P: 0.25, 100001 NP: 0.5 P: 0, 100000 NP: 0.25 P: 0]
[100003 NP: 0 P: 250.25, 100001 NP: 0.5 P: 0, 100441 NP: 0.25 P: 0.25, 100000 NP: 0.25 P: 0]
[100003 NP: 0 P: 250.25, 100001 NP: 0.5 P: 0, 100101 NP: 0.25 P: 0.25, 100000 NP: 0.25 P: 0]

2^1 [100021 NP: 0.5 P: 0, 100000 NP: 0.25 P: 0]

2^10 [100153 NP: 0 P: 2.75, 100001 NP: 0.5 P: 0, 100000 NP: 0.25 P: 0]

2^100 [100153 NP: 0 P: 25.25, 100001 NP: 0.5 P: 0, 100000 NP: 0.25 P: 0]

2^1000

[100003 NP: 0 P: 250.25, 100001 NP: 0.5 P: 0, 100651 NP: 0.25 P: 0.25, 100000 NP: 0.25 P: 0]

2^10000

[100003 NP: 0 P: 2500.25, 100001 NP: 0.5 P: 0, 100471 NP: 0.25 P: 0.25, 100000 NP: 0.25 P: 0]

The probabilities above were obtained by running the program MillerR.java several times, each time had a trial of 1, 10, 100, 1000, and 10000 random a's to determine whether the number is prime or a composite. The biggest decimal value a group was picked along with its lowest integer value. To better explain the above output integers whose p(prime counter) is 2500.25 are definitely primes since not even once it was count as a composite number. Integers whose p(prime counter) is 0 and NP(not prime counter) is 0.5 or 0.25 are definitely composite since not even once it was told to be a prime. However, during running the MillerR.java program we were able to see that some number came into the category of "MAYBE" a composite number since NP = P = 0.25 we do not really know what the number is because according to the algorithm it could be either prime or a composite number.

- **Also submit a hardcopy of your code, and a clear description of what you did.**

Description:

To be able to do the above test we had to create a program that test for each one of the given integers, in order to do so we needed to build three main methods:

First, create a method named `modExp(a, m, n)`. This method takes a random integers less than n , the integer that is being tested, to multiply itself m times. We did so by using the modular exponentiation algorithm provided in class.

Second, create a method named `is_pPrime(int n)`. This method takes an integer as a parameters, this is integer is the number to be tested. After finding k , where $2^{km} \equiv 1 \pmod n$ and m is odd. This method first tests for all base cases to check whether or not the number is a composite. First, if the number is less or equal to 1 n is a composite. Second, if the number is equal or less than 2 is a prime. Third, if the number is divisible by 2 is a composite. If none of the above cases are met then, n goes into a loop from 0 to t where t is equal to 1000 in this program and a random number “ a ” is generated each time it loops. For each loop, each “ a ” is saved along with the answers of whether the number is prime or not. Each loop “ a ” calls the third function named `is a witness` to check and store whether the number is a prime or not.

Third, create a method named `checkWitness(a, n)`. This method takes the random generated a along with n , the number to be tested, to check whether or not the number is a prime or not.

Finally, after all possibilities are stored in the following manner: Each n , number to be tested, has its own list of random a 's for better analysis and its own prime and not prime counters. Once the test is done, we proceeded to sort all numbers in groups for example; all numbers with the same probability decimal values of 250.25 are sorted in one single group, and the lowest number n was picked. It was done the same way for the others numbers.

Concluding, by analyzing the results in the output file generated you can easily recognized those numbers that are definitely prime and those that are definitely composite. Nevertheless, minority (less than three numbers) of numbers fell into a “MAYBE” in which the algorithm was not able to determine whether they were prime or composite since NP (not prime counter) and P (prime counter) had the same number of probability.

To compute modular exponentiation, you must implement the square-and-multiply algorithm yourself. Two versions of this algorithm can be found in Exercise 3.23 (page 107/108) and we went over another version in class.

```
private static long modExp(long a, long e, long n) {
    long result = 1;
    long temp = a;
    while (e > 0) {

        if (e % 2 == 1)
            result = (result * temp) % n;

        temp = (temp * temp) % n;
        e /= 2;
    }
    return result;
}
```

Example Output: For a better reference please look at output.txt

Is 100000 Prime ? false	%Counter P:250.25
Not Prime Counter (NP)C: 1.0	
Prime Counter (P)C:0.0	A's (78326)
%Counter NP: 0.25	Not Prime Counter: 0
%Counter P:0	Prime Counter :1
Is 100001 Prime ? false	A's (26484)
Not Prime Counter (NP)C: 2.0	Not Prime Counter: 0
Prime Counter (P)C:0.0	Prime Counter :1
%Counter NP: 0.5	
%Counter P:0	A's (38271)
A's (32558)	Not Prime Counter: 0
Not Prime Counter: 2	Prime Counter :1
Prime Counter :0	A's (30395)
Is 100002 Prime ? false	Not Prime Counter: 0
Not Prime Counter (NP)C: 1.0	Prime Counter :1
Prime Counter (P)C:0.0	A's (61394)
%Counter NP: 0.25	Not Prime Counter: 0
%Counter P:0	Prime Counter :1
Is 100003 Prime ? true	
Not Prime Counter (NP)C: 0.0	A's (39660)
Prime Counter (P)C:1001.0	Not Prime Counter: 0
%Counter NP: 0	Prime Counter :1

A 1000 A's later for each number

[100003 NP: 0 P: 250.25, 100019 NP: 0 P: 250.25, 100043 NP: 0 P: 250.25, 100049 NP: 0 P: 250.25, 100057 NP: 0 P: 250.25, 100069 NP: 0 P: 250.25, 100103 NP: 0 P: 250.25,]

100001 NP: 0.5 P: 0, 100005 NP: 0.5 P: 0, 100007 NP: 0.5 P: 0, 100009 NP: 0.5 P: 0, 100011 NP: 0.5 P: 0, 100013 NP: 0.5 P: 0, 100015 NP: 0.5 P: 0, 100017 NP: 0.5 P: 0,]

100651 NP: 0.25 P: 0.25,]

100000 NP: 0.25 P: 0, 100002 NP: 0.25 P: 0, 100004 NP: 0.25 P: 0, 100006 NP: 0.25 P: 0, 100008 NP: 0.25 P: 0, 100010 NP: 0.25 P: 0, 100012 NP: 0.25 P: 0, 100014 NP: 0.25 P: 0,]

highest Probabilities According to its Percentage and Lowest Value:

[100003 NP: 0 P: 250.25, 100001 NP: 0.5 P: 0, 100651 NP: 0.25 P: 0.25, 100000 NP: 0.25 P: 0]

Prime Numbers According to highest Probabilities :

100003	100333	100559
100019	100343	100591
100043	100357	100609
100049	100361	100613
100057	100363	100621
100069	100379	100649
100103	100391	100669
100109	100393	100673
100129	100403	100693
100151	100411	100699
100153	100417	100703
100169	100447	100733
100183	100459	100741
100189	100469	100747
100193	100483	100769
100207	100493	100787
100213	100501	100799
100237	100511	100801
100267	100517	100811
100271	100519	100823
100279	100523	100829
100291	100537	100847
100297	100547	100853
100313	100549	]

Example Code: For a better reference please look and run MillerR.java and A.java

```
// witness n's compositeness
public boolean checkWitness(long a, long n) {

// get t and u such that p-1 = 2^k * m, m is odd
    int k = 1;
    long m = (n-1)/2;
    while (m%2 == 0) {
        m = m/2;
        k++;
    }

//check if a is a witness to n's compositeness
    long b = modExp(a, m, n);
    for (int i = 0; i < k; i++) {
        long bnew = (b*b)%n;
        if (bnew == 1 && !(b == 1)
            && !(b == (n - 1))) {
            // System.out.println(1);

            prime++;
            this.a.get(counter).prime++;
            return true;
        }
        b = bnew;
    }
    if (b == 1) {
        prime++;
        this.a.get(counter).prime++;
        // System.out.println(1);
    } else {
        nPrime++;
        this.a.get(counter).nPrime++;
    }
}

return !(b==1);
}

// is n probably prime
public boolean is_pPrime(long n) {
    // simple cases: 0, 1, 2, or even
    if (n <= 1 || (n%2 == 0)) {
        // System.out.println(0);
        nPrime++;
        return false;
    } else if (n <= 3) {
        // System.out.println(1);
        prime++;
        return true;
    } else {
        // repeat 10000 times for error 2^10000
        for (int i = 0; i < 1000; i++) {
            // random integer in (1, n-1)
            long a = 0;
            do {
                a = new
                    Random().nextInt((int) (n-1));
            } while (a <= 1 || a >= (n-1));

            as = new A(a);
            this.a.add(as);
            counter++;
            isPrime.add(a);
            // check if it's a witness
            if (checkWitness(a, n)) {
                // System.out.println(0);
                this.a.get(counter).nPrime++;
                nPrime++;
                return false; // definitely
                    composite
            }
        }
        // System.out.println(1);
        this.a.get(counter).prime++;
        prime++;
        return true;
    } // probably prime
}
```